

UNIT-IV

Dynamic Programming:

- similar to divide & conquer. (prob → divided to subprobs)
- solns of subproblems are stored in a table.
- to solve a prob: 1. optimal substructure
- 2. recurrence
- 3. approach $\begin{cases} \rightarrow \text{top down} \\ \rightarrow \text{bottom down} \end{cases}$

Binomial - Coefficient:

The objective of B.Coeff is to find the no. of combinations of 'k' elements in a set of 'n' elements.

It is denoted as $C(n, k)$ or nC_k

$${}^nC_k = \frac{n!}{k! (n-k)!} \quad \text{and} \quad C(n, k) = \begin{cases} 1 & \text{if } k=0 \text{ or } k=n \\ 0 & \text{if } k > n \\ C(n-1, k) + C(n-1, k-1) & \text{if } k < n \text{ or } k \neq 0 \end{cases}$$

set
ex ① $n=4, k=3$

$${}^nC_k = {}^4C_3 = \frac{4!}{3! \times 1!} = 4.$$

Table is const^d with $n+1$ rows & $k+1$ columns.

	$k+1$			
	0	1	2	3
0	1			
1	1	1		
2	1	2	1	
3	1	3	3	1
4	1	4	6	4

4C_3 is 4.

ex 2) $n=6, k=4$

$${}^6C_4 = 15 //$$

	0	1	2	3	4
0	1				
1	1	1			
2	1	2	1		
3	1	3	3	1	
4	1	4	6	4	1
5	1	5	10	10	5
6	1	6	20	20	15

ALGORITHM Binomial coeff (n, k)

// computes $C(n, k)$ by dynamic programming

// input: two non-negative integers n & k

// output: value of $C(n, k)$

for $i \leftarrow 0$ to n do

for $j \leftarrow 0$ to $\min(i, k)$

if $j = 0$ or $j = i$

$C[i, j] = 1$

else

$C[i, j] = C[i-1, j] + C[i-1, j-1]$

return $C[n, k]$

Time Complexity:

$$T(n) = \sum_{i=0}^n \sum_{j=0}^k 1$$

$$= \sum_{i=0}^n k = O(nk)$$

Recurrence relⁿ

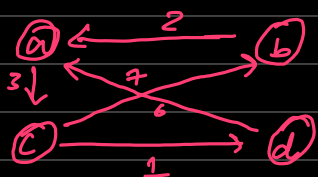
$A(n, k) = 0$ when $k = 0$ or $k = n$

$A(n, k) = A(n-1, k) + A(n-1, k-1) + 1$; otherw^{ise}

Floyd's Alg:

⇒ goal: all pairs shortest path Alg.

⇒ input: a weighted graph. (D)



(find shortest path from each node to the other)

step 0
⇒ Construct a matrix 4×4 since $n = 4$.

	a	b	c	d	
a	0	∞	3	∞	0 ⇒ self
b	2	0	∞	∞	∞ ⇒ no connec
c	∞	7	0	1	
d	6	∞	∞	0	

= D

step 2 ⇒ There are 4 nodes i.e Floyd's alg will have 4 iterations: D_1, D_2, D_3, D_4 .

step 3 ⇒ For each iteration, take each row & each column respectively. Output of last iteration will be shortest path.

D^1	0	∞	3	∞	↔ (1st row, 1st col)
0	0	∞	3	∞	↔ (1st row & 1st column remains same as the 'D' graph)
2	2	0	5	∞	2 + 3 = 5 (min) ✓ prev = ∞
∞	∞	7	0	1	2 + ∞ = ∞ prev = ∞
6	6	∞	9	0	

Hint: if prev has 0s then put all 0s in next iteration also.

D^2	2	0	5	∞	↔ (2nd row, 2nd col of D^1)
∞	0	∞	3	∞	↔ (compare with prev table & get min)
0	2	0	5	∞	
7	9	7	0	1	7 + ∞ = ∞ prev = 1
∞	6	∞	9	0	

Hint: if ∞ & ∞ is being compared, directly take prev value.

D^3	9	7	0	1	
3	0	10	3	4	
5	2	0	5	6	
0	9	7	0	1	
9	6	16	9	0	

D^4	a	b	c	d
6	6	16	9	0
4	0	10	3	4
6	2	0	5	6
1	7	7	0	1
0	6	16	9	0
i				

ALGORITHM Floyd's ($W[1..n, 1..n]$)

// implements Floyd's using dynamic prog.

// input: weighted graph i.e a matrix.

// output: matrix i.e all pairs shortest path

$D \leftarrow W$

for $i \leftarrow 1$ to n do

for $j \leftarrow 1$ to n do

for $k \leftarrow 1$ to n do

$D[i, j] \leftarrow \min \{ D[i, j], D[i, k] + D[k, j] \}$

return D

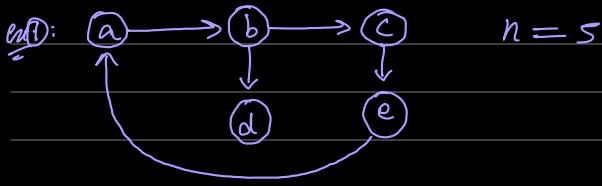
Time complexity:

Basic op ⇒ addition

$O(n^3)$

Warshall's alg:

- used to compute transitive closeness of a directed graph (not weighted graph).
- T.C of a directed graph with 'n' vertices can be defined as an 'n x n' boolean matrix.



input

	a	b	c	d	e
a	0	1	0	0	0
b	0	0	1	1	0
c	0	0	0	0	1
d	0	0	0	0	0
e	1	0	0	0	0

self $\Rightarrow 0$
no connec $\Rightarrow 0$

direct output method

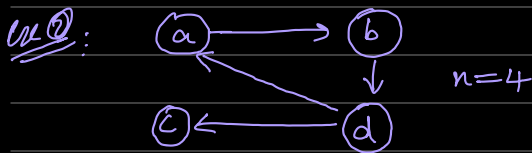
output

	a	b	c	d	e
a	1	1	1	1	1
b	1	1	1	1	1
c	1	1	1	1	1
d	0	0	0	0	0
e	1	1	1	1	1

(in-directed path)
if there exists a non-trivial directed path $\Rightarrow ①$
if no path exists in the directed path $\Rightarrow ②$

~> [as we cannot reach any other node from 'd']

How to trace ???



A

input

	a	b	c	d
a	0	1	0	0
b	0	0	0	1
c	0	0	0	0
d	1	0	1	0

(4 iterations)

Step ①: '0' $\Rightarrow$ as it is $\Rightarrow$ prev val.
Step ②: AND operation within matrix
Step ③: OR operation with prev $\Rightarrow$ Ans

R¹

	a	b	c	d
a	0	1	0	0
b	0	1	0	0
c	0	0	0	1
d	0	0	0	0
1	1	1	1	0

Hint: if '0' is reference, no-write the prev values in that particular row or column.

Basically,

if reference is '0' $\Rightarrow$ as it is
'1' $\Rightarrow$ AND op
 $\Rightarrow$ OR op $\Rightarrow$ ans

R^2		R^3		R^4	
	0 0 0 1		0 0 0 0		0 0 0 0
1	0 1 0 1	0	0 1 0 1	0	1 1 1 1
0	0 0 0 1	0	0 0 0 1	0	1 1 1 1
0	0 0 0 0	0	0 0 0 0	0	0 0 0 0
1	1 1 1 1	1	1 1 1 1	0	1 1 1 1

(we can also do it directly and check that it is correct)

ALGORITHM warshall ($A[1..n, 1..n]$)

// implements warshall alg to compute transitive closure.

// I/O: adjacency matrix A of directed graph with 'n' vertices.

// O/P: Transitive closure of graph.

$R^0 \leftarrow A$

for $i \leftarrow 1$ to n do

for $j \leftarrow 1$ to n do

for $k \leftarrow 1$ to n do

$R^k[i, j] = R^{k-1}[i, j] \text{ or } R^{k-1}[i, k] \text{ and } R^{k-1}[k, j]$

return R^n

Time Complexity:

$O(n^3) \Rightarrow 3$ for loops //

Diff b/w Dynamic Prog and Divide and Conquer:

Divide & Conquer

- 'div', 'conquer', 'combine' approach.
- breaks problems into smaller subproblems.
- no shared info b/w subproblems.
- Subproblems are solved sequentially.
- Typically uses recursion.
- high mem overhead.
- generally has higher time complexity.

ex: binary search etc.

Dynamic Programming

- 'divide', 'solve', 'combine' approach.
- breaks the problem into overlapping subproblems.
- stores solns in a table for future reference.
- subproblems are solved in any order.
- Typically uses iteration.
- low mem overhead.
- generally has low time complexity.

ex: knapsack etc.

0-1 Knapsack Problem:

→ In the knapsack prob, there will be n objects of known weights denoted by $w_1, w_2, w_3, \dots, w_n$
profits denoted by $v_1, v_2, v_3, \dots, v_n$

→ knapsack capacity is denoted by W .

→ Aim: add more profitable objects into the knapsack with a constraint that the weight of the added obj's should not exceed the knapsack capacity.

→ let x_i denote whether the obj is included or not. $\left. \begin{array}{l} \text{if } x_i = 1; \text{ obj is included} \\ \text{if } x_i = 0; \text{ obj is not included} \end{array} \right\} \therefore 0-1 \text{ knapsack}$

→ knapsack prob can be denoted as:

$$\text{Maximize } \sum_{i=1}^n x_i v_i //$$

Ex: (By Brute force method) n obj's = 3; ^{weights} $w = 2, 3, 2$; ^{profits} $pt = 5, 10, 8$ and ^{knapsack capacity} $W = 5$

Obj 1 $\rightarrow 5$
2 $\rightarrow 10$
3 $\rightarrow 8$

does not exceed knapsack capacity

1, 2 $\rightarrow 15$ } $2+3=5 \leq 5$

1, 3 $\rightarrow 13$ } $2+2=4 \leq 5$

2, 3 $\rightarrow 18$ } $3+2=5 \leq 5$ max profit obtained = 18.

Using dynamic programming:

→ we construct a table of $n+1$ rows ($n = \text{no. of obj's}$) $0 \dots n$ and $W+1$ columns ($W = k/s$ capacity)

→ let the table be denoted by V , the entries in V are cal as:

$$\begin{cases} V[i-1, j] & ; \text{ if } j - w_i < 0 \\ \max \{ V[i-1, j], V[i-1, j - w_i] + v_i \} & ; \text{ if } j - w_i \geq 0 \end{cases}$$

Ex: $n = 4$, $W = 5$, ^(w_i) weights = $\{2, 1, 3, 2\}$, ^(v_i) profits = $\{8, 6, 16, 11\}$.

		^{knapsack capacity}					
		0	1	2	3	4	5
objects	0	0	0	0	0	0	0
	1	0	0	8	8	8	8
	2	0	6	8	14	14	14
	3	0	6	8	16	22	24
	4	0	6	11	17	22	27

[1st row & 1st col = 0]

[1st row can have vals equal to 8 only, since the profit of placing the 1st obj is 8].

[2nd row can have vals equal to 8, 6 or 14 as the profit of placing 2nd obj can be 1, 2 or 1, 2]

[Similarly for 3rd row the max entry can be $8 + 6 + 16 = 30$ only]

→ $V[1, 1] = j - w_i = 1 - 2 = -1 < 0$ (comes under 1st category) $\Rightarrow$ insert val on prev row i.e. 0 //

→ $V[1, 2] = j - w_i = 2 - 2 = 0 \geq 0$ (second category) $\Rightarrow$ find $\max\{V[0, 2], V[0, 0] + 8\} \Rightarrow \max\{0, 0 + 8\} = 8 //$

→ $V[2, 1] = j - w_i = 1 - 1 = 0 \geq 0$ (2nd) $\Rightarrow \max\{V[1, 1], V[1, 0] + 6\} \Rightarrow \{0, 0 + 6\} = 6 //$ (Similarly for others)

$$\Rightarrow v[2,2] \Rightarrow j-w_i \Rightarrow 2-1 = 1 \geq 0 \Rightarrow \max \{ v[1,2], v[1,1] + 6 \} \Rightarrow \{ 8, 0+6 \} = 8 //$$

$$\therefore v[n, w] \quad \leftarrow \text{max profit}$$

$$v[4,5] = 27$$

$\Rightarrow$ how to determine the obj in knapsack? (By Bottom up approach).

$\Rightarrow$ go from last i.e 27 and one prev to that i.e 24 i.e $v[3,5]$. They are not equal
 $\therefore v[4,5] \neq v[3,5]$

So, the 4th obj shd be included.

$\Rightarrow$ do the same for $v[3,5-2] = v[3,3]$

$$v[3,3] = v[2,3] \quad \text{prev row value}$$

$\therefore$ 3rd obj shd be included //

$$\Rightarrow v[2,3-3] = v[2,0]$$

$\hookrightarrow$ Stop when it comes to 0 //

$\therefore$ Obj included are 3rd & 4th obj with weights $3+2=5=w //$

Memory Func Knapsack Problem:

advantage

$\Rightarrow$ In this approach, it was required to find all vals like $v[4,0]$ $v[4,1]$ $v[4,2]$ etc as we need only the last vals for Bottom up approach. We did additional computations.

$\Rightarrow$ These additional computations can be reduced by Mem Func method.

$\Rightarrow$ mem func are recursive in nature. It uses Bottomup and topdown approach, both.

ex: $n=4, w=2, 3, 2$
 $W=5, v=8, 6, 10 //$

[initialize all vals to -1]

[what max profit is at the last position, we need to cal ones for last]

[solve all to the Alg in next page]

[start from $v[4,5]$ and find all vals]

	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	-1	-1	-1	-1	-1
2	0	-1	-1	-1	-1	-1
3	0	-1	-1	-1	-1	-1
4	0	-1	-1	-1	-1	-1

$i=4$
 $j=5$

step 1 $\Rightarrow$ check $[i,j] < 0$.

step 2 $\Rightarrow$ if yes, check $j < w + [i]$.

step 3 $\Rightarrow$ if yes, $v[i,j] = v[i-1,j]$.

step 4 $\Rightarrow$ if not, $v[i,j] = \max(v[i-1,j], v[i-1,j-w[i]] + v[i])$

$$v[4,5] = \text{MFKP}$$

$$v[i,j] < 0, \text{ yes}$$

$$j < w + [i], \text{ no}$$

$$\therefore v[4,5] = \max(v[3,5], v[3,3] + 11) \Rightarrow \text{we dont know } v[3,5] \text{ and } v[3,3]$$

$$v[3,5] = \text{MFKP}$$

$$v[3,3] = \max(v[2,3], v[2,2] + 10)$$

$$v[3,5] < 0, \text{ yes}$$

$$5 < 3, \text{ no} \Rightarrow v[3,5] = \max(v[2,5], v[2,2] + 10)$$

[11th cal for (2,5) (2,3) (2,0)]

$$V[2,5] = \max(V[1,5], V[1,4] + 6)$$

$$V[2,2] = \max(V[1,2], V[1,1] + 6)$$

$$V[2,3] = \max(V[1,3], V[1,2] + 6)$$

$$V[2,0] = V[1,0] = 0 \quad \left[\begin{array}{l} V[2,0] = \text{MFKP} < 0, \text{yes} \\ 0 < 1, \text{yes} \Rightarrow V[1,0] \Rightarrow 0 \end{array} \right]$$

$$V[1,5] = \max(V[0,5], V[0,3] + 8)$$

$$V[1,4] = \max(V[0,4], V[0,2] + 8)$$

$$V[1,3] = \max(V[0,3], V[0,1] + 8)$$

$$V[1,2] = \max(V[0,2], V[0,0] + 8)$$

$$V[1,1] = V[0,1] = 0$$

$$= \max(8, 8+6) = 14 //$$

$$= \max(8, 0+6) = 8 //$$

$$= \max(8, 8+6) = 14 //$$

$$= 0 //$$

$$= \max(0, 0+8) = 8 //$$

$$= \max(0, 0+8) = 8 //$$

$$= \max(0, 0+8) = 8 //$$

$$= \max(0, 0+8) = 8 //$$

	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	-1	8	8	8	8
2	0	-1	8	14	-1	14
3	0	-1	-1	16	-1	24
4	0	-1	-1	-1	-1	27

Alg used for filling the table: Mem func Alg:

Algorithm MemFunc knapsack (i, j)

// implements mem func method to solve the knapsack prob.

// input: two non-negative integers: i and j
 i $\Rightarrow$ no. of first obj.
 j $\Rightarrow$ knapsack capacity.

// output: the val of an optimal feasible subset of first i items.

1st row $\leftarrow 0$, 1st col $\leftarrow 0$, remaining $\leftarrow -1$.

if $V[i, j] < 0$

if $j < \text{wt}[i]$

value $\leftarrow \text{MFKP}(i-1, j)$

else

value $\leftarrow \max(\text{MFKP}(i-1, j), \text{MFKP}(i-1, j - \text{wt}[i]) + V[i, j])$

$V[i, j] \leftarrow \text{value}$

return $V[i, j]$

Greedy Technique:

$\rightarrow$ feasible (The prob shd satisfy the prob constraints)

$\rightarrow$ locally optimal (it has to be the best choice among all feasible solns).

$\rightarrow$ irrevocable (decision made cannot be changed)

$\Rightarrow$ general alg for soln a problem using greedy technique:

Algorithm greedy (A, n)

// solves a prob using greedy method

// A is the input, which is a list of n elements

Solution $\leftarrow \phi$

for $i \leftarrow 1$ to $n-1$

$x = \text{choose}(A)$

if (feasible(x))

Solution = Union (Solution, x)

return solution

psim's alg
dijkstra's alg
factorial k-s prob

Prim's Alg: to construct a Min Spanning Tree

→ goal is to construct a MST → min spanning tree.

→ A spanning tree of a connected graph is a acyclic subgraph which has all the vertices of the graph. Many no. of Spanning trees can be made for a graph.

→ M.S.T is the one with min weight.

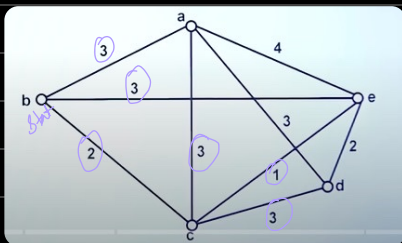
→ This M.S.T can be found by Prim's Alg.

step 1 select any vertex & choose edge and smallest weight from it.

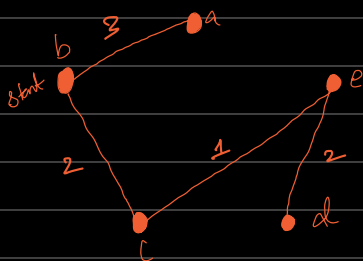
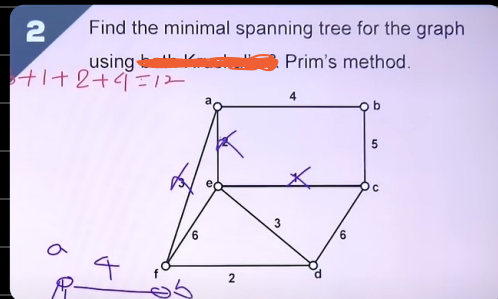
step 2 at each step, choose edge of smallest weight, joining a vertex already included to vertex not yet included

step 3 Continue until all vertices are included.

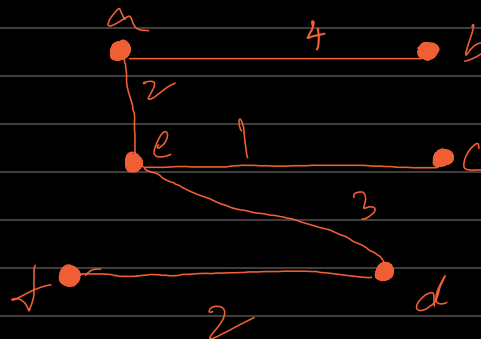
ex 1



opt: (2)



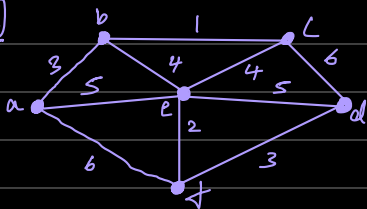
$$\Rightarrow \text{weight} = 3 + 2 + 1 + 2 = 8 //$$



$$\Rightarrow 1 + 2 + 2 + 3 + 4 = 12 = \text{weight} //$$

→ write it like this in exam:

ex 3



no. of vertices = 6

no. of iterations = 6 - 1 = 5

→ $V = \{a, b, c, d, e, f\}$

→ $E_T = \emptyset$ (shd contain all edges used in the const of MST)

} 2 sets are taken for Vertices & edges.

1st iteration: $V_T = \{a\}$ → selected vertex
 $V - V_T = \{b, c, d, e, f\}$ → remaining vertices
 (a,b) (3), (a,c) (5), (a,e) (6) → edges present from selected vertex to the $V - V_T$ set.
 $E_T = \{(a,b)\}$ → add the min edge to edge set

2nd iteration

$$V_T = \{a, b\}$$

$$V - V_T = \{c, d, e, f\}$$

$$(a, e) \quad (a, f) \quad (b, e) \quad (b, c)$$

5 6 4 1

$$E_T = \{(a, b), (b, c)\}$$

3rd iteration

$$V_T = \{a, b, c\}$$

$$V - V_T = \{d, e, f\}$$

$$(a, e) \quad (a, f) \quad (b, e) \quad (c, d) \quad (c, e)$$

5 6 4 6 4

$$E_T = \{(a, b), (b, c), (b, e)\}$$

2 same min costs out present, we 1st one that arrives.

4th iteration

$$V = \{a, b, c, e\}$$

$$V - V_T = \{d, f\}$$

$$(a, f) \quad (c, d) \quad (e, d) \quad (e, f)$$

6 6 5 2

$$E_T = \{(a, b), (b, c), (b, e), (e, f)\}$$

5th iteration

$$V = \{a, b, c, e, f\}$$

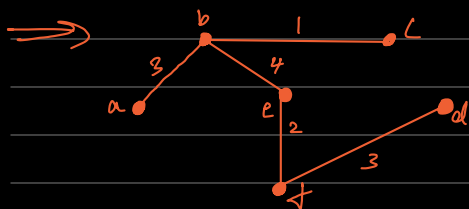
$$V - V_T = \{d\}$$

$$(c, d) \quad (e, d) \quad (f, d)$$

6 5 3

$$E_T = \{(a, b), (b, c), (b, e), (e, f), (f, d)\}$$

3 1 4 2 3



$$\Rightarrow \text{weight} = 3 + 1 + 4 + 2 + 3 = 13 //$$

Algorithm primes (G_0)

// output: E_T i.e the set of edges used in construct of MST.

$V_T = \{v_0\}$ // v_0 is the start (source) vertex.

$E_T = \emptyset$

for $i \leftarrow 1$ to $|V| - 1$

find a min weight edge $e^* = (v^*, u^*)$ among all the edges (v, x) such that v is in V_T and u is in $V - V_T$

$V_T \leftarrow V_T \cup u^*$

$E_T \leftarrow E_T \cup \{e^*\}$

return E_T

Time Complexity:

$\Rightarrow$ if the input graph G is rep as a weighted matrix, efficiency = $\underline{\Theta(|V|^2)}$

$\Rightarrow$ if the input graph G is rep as an adjacency list & a queue is implemented as a min-heap the running time is $\underline{O(|E| \log |V|)}$.

$\therefore$ we need to perform $|V| - 1$ deletions and makes $|E|$ notifications and changes the heap

$$\Rightarrow \underline{O(\log |V|)}$$

$$\therefore \underbrace{(|V| - 1)}_{\text{higher order}} + \underbrace{|E| \log |V|}_{\text{higher order}} = \underline{O(|E| \log |V|)}$$

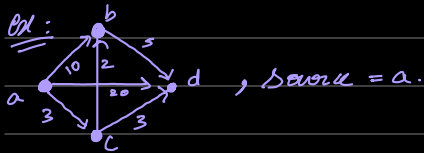
Dijkstra's Alg: To find the shortest path.

⇒ Based on greedy method.

⇒ goal ⇒ to find the shortest path from a source in a weighted graph.

⇒ aka single source shortest path algorithm.

⇒ all edges in dij alg shd have the values.



$$V = \{a, b, c, d\}$$

$$W = \{a, b, c, d\}$$

vertex	(dist) $d[]$	(parent) $p[]$
a	0	nil
b	∞	nil
c	∞	nil
d	∞	nil

$$\text{soln} = \emptyset$$

1st iteration

$$W = \{a, b, c, d\}$$

vertex	$d[]$	$p[]$
a	0	Nil
b	10	a
c	3	a
d	20	a

$$\text{soln} = \{a\}$$

among $W = \{a, b, c, d\}$, a has the least value.

2nd iteration

$$W = \{b, c, d\}$$

vertex	$d[]$	$p[]$
a	0	Nil
b	5	c
c	3	a
d	6	c

$$\text{soln} = \{a, c\}$$

among $W = \{b, c, d\}$, c has the least value.

3rd iteration

$$W = \{b, d\}$$

vertex	$d[]$	$p[]$
a	0	Nil
b	5	c
c	3	a
d	6	c

$$\text{soln} = \{a, c, b\}$$

as it is

smallest val is b

$$5+6=11 < 20$$

$$2+5=7 < 10$$

$$3+3=6 < 20$$

(c dist will not change)

4th iteration

$$W = \{d\}$$

vertex id	d[]	p[]
a	0	Nil
b	5	c
c	3	a
d	6	c (d does not have outgoing edges)

$$Sourcen = \{a, c, b, d\}$$

Algorithm Dijkstra (G, s)

Output: The shortest path from s to all vertices in V

for each vertex v in V do

$$d[v] \leftarrow \infty$$

$$p[v] \leftarrow Nil$$

$$d[s] \leftarrow 0$$

$$Sourcen \leftarrow \emptyset$$

$$W \leftarrow V$$

while $W \neq \emptyset$

$$u \leftarrow \text{extract_min}(W)$$

$$Sourcen \leftarrow Sourcen \cup \{u\}$$

$$W \leftarrow W - \{u\}$$

for each vertex v in V adjacent to u

$$\text{if } d[v] > d[u] + c(u, v)$$

$$d[v] \leftarrow d[u] + c(u, v)$$

$$p[v] \leftarrow u$$

return d

Time Complexity:

$\Rightarrow$ If G is rep as adjacency matrix $\Rightarrow \underline{\underline{|V|^2}}$

$\Rightarrow$ If G is rep as an adjacency list and min heap is created for all the edges $\Rightarrow \underline{\underline{O(V+E)}}$

$\Rightarrow$ extract min to $s \Rightarrow \underline{\underline{O(\log V)}}$

$$\therefore O[(V+E) \times \log V] = O[V \log V + \underbrace{E \log V}_{\text{higher order}}] = \underline{\underline{O(E \log V)}}$$

Huffman Coding:

⇒ used for data encoding

- used for data compression
- used for data integrity

⇒ encoding can be done in 2 diff ways: ① fixed length encoding.
② variable length encoding.

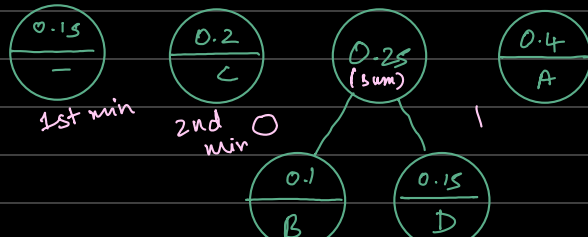
⇒ we build a Binary Tree which gives the codeword for the characters

Step ① Initialize n (no. of char^s to be encoded) 0-1 tree (binary tree - left child marked with 0 & right child marked with 1) and label with the appropriate char^s as well as record the freq of that char^s in the tree root to indicate the tree weight.

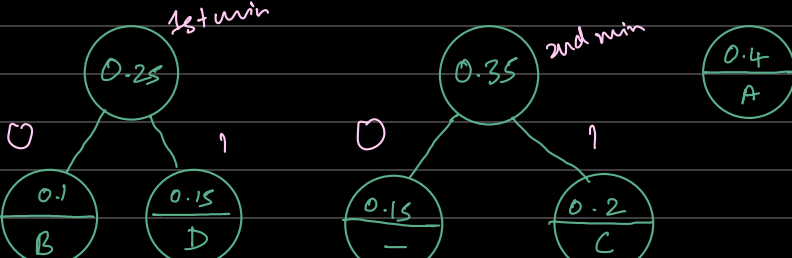
Step ② repeat the foll until a single tree is obtained.
→ find 2 trees with smallest weight and create a 0-1 tree with the smallest as left child and second smallest as right child. Record the sum of weights in the root of the new tree.

Q)  20 characters
 $A = 8 = 40\%$ ($\frac{8}{20} \times 100$)
 $B = 2 = 10\%$
 $C = 4 = 20\%$
 $D = 3 = 15\%$
 $- = 3 = 15\%$
 , draw H. Tree, find compression ratio, encode BAD-DAA

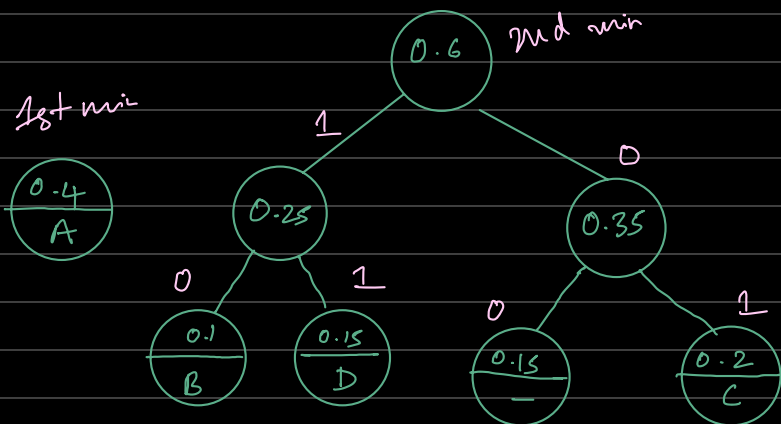
Step ①  → trees of char^s.
 → char^s to be encoded.
 1st min: B (0.1)
 2nd min: D (0.15)

Step ② 
 1st min: - (0.15)
 2nd min: (0.25) sum

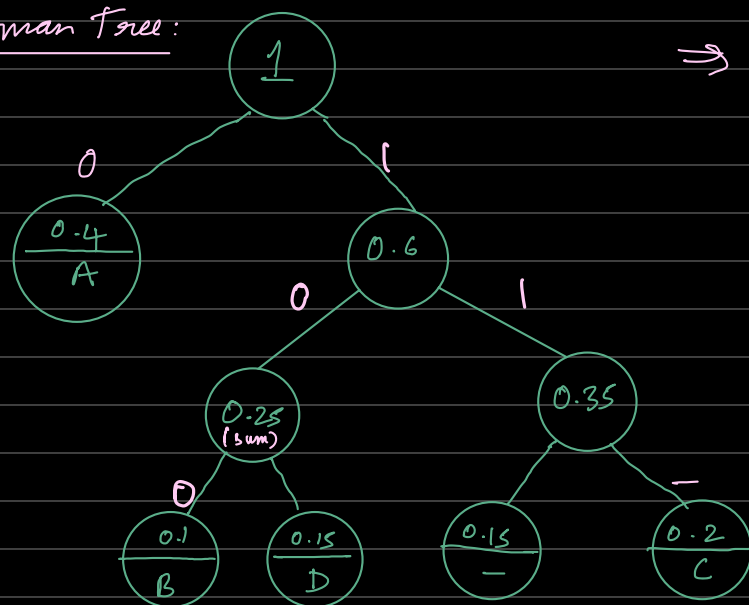
arrange in increasing order in each step.


 1st min: (0.35) sum
 2nd min: C (0.2)

(Consider the root of tree also while finding 1st & 2nd min)



⇒ Huffman Tree:



⇒ encoded vals: (from root)

$$\begin{aligned} A &= 0 \quad (\text{length} = 1) \\ B &= 100 \\ C &= 111 \\ D &= 101 \\ - &= 110 \end{aligned} \quad \left. \vphantom{\begin{aligned} A \\ B \\ C \\ D \\ - \end{aligned}} \right\} (\text{length} = 3)$$

⇒ avg no. of bits per char = $(1 \times 0.4) + (3 \times 0.1) + (3 \times 0.2) + (3 \times 0.15)$

$$= 0.4 + 0.3 + 0.6 + 0.45 + 0.45$$

$$= 2.2 //$$

⇒ Compression ratio = $\left(\frac{3 - 2.2}{3} \right) \times 100$

$$= 26\% //$$

⇒ encode BAD-DAA: $\underline{100010111010100}$

B A D - D A A

Fractional Knapsack Problem: (using Branch & Bound)

⇒ The objective of k/s prob is to maximize the profit, hence it is an upper bound funⁿ.

⇒ Here, we shd always order the items in the descending order of their profits to weight ratio

i.e $\frac{V_1}{W_1} \geq \frac{V_2}{W_2} \geq \frac{V_3}{W_3} \geq \dots \geq \frac{V_n}{W_n}$

⇒ The upperbound of the knapsack prob is cal

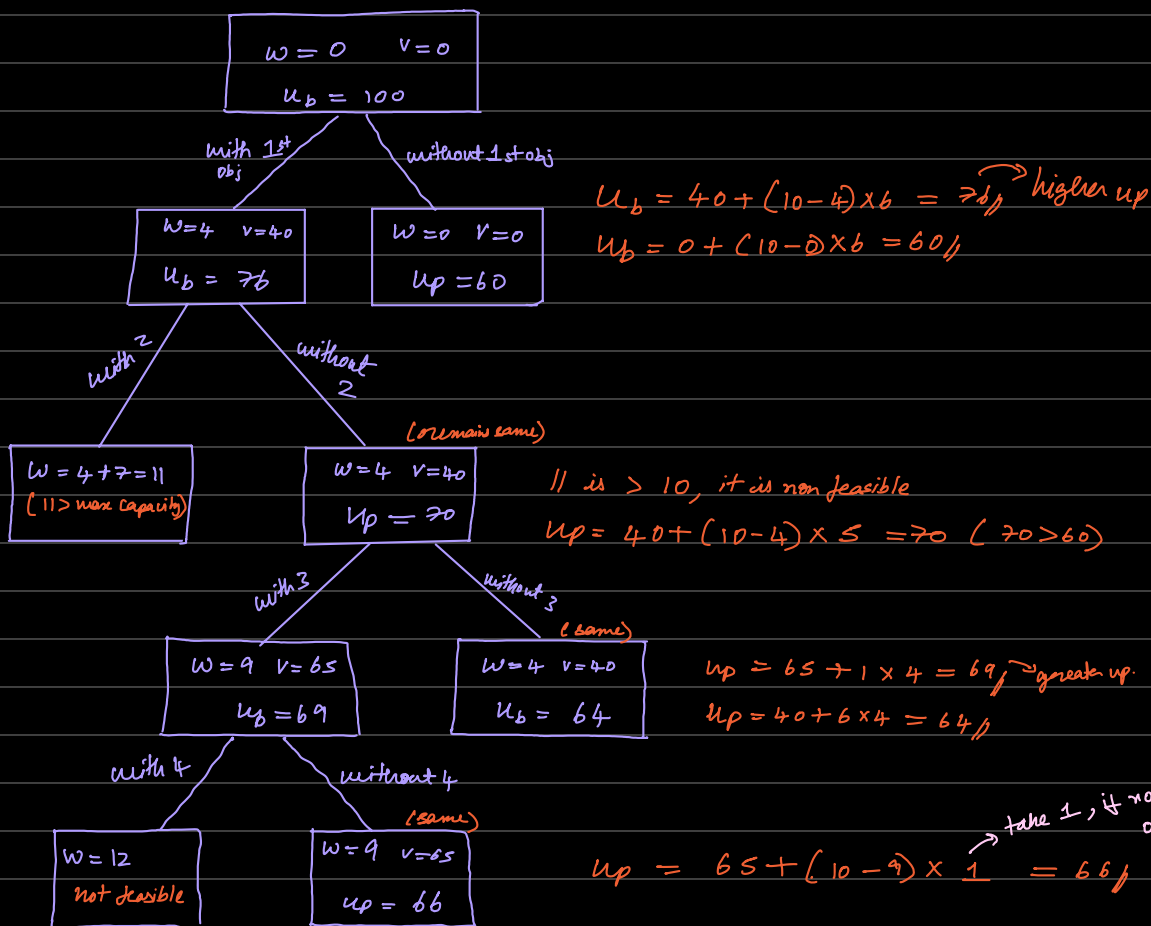
$$U_b = r + (M - w) \frac{V_{i+1}}{W_{i+1}}$$

Q) $n=4$, $M=10$, $W=4\ 7\ 5\ 3$, $V=40\ 42\ 25\ 12$

decending order only

$$\Rightarrow \frac{V}{W} = 10\ 6\ 5\ 4 \Rightarrow u_b = 0 + (10-0) \times 10 \xrightarrow{\text{next obj } \frac{v_{i+1}}{w_{i+1}}} = 0 + 10 \times 10 = 100 \quad (\text{when } v=0, w=0)$$

Obj no. = 1 2 3 4
 $W = 4\ 7\ 5\ 3$
 $V = 40\ 42\ 25\ 12$
 $\frac{V}{W} = 10\ 6\ 5\ 4$



$\Rightarrow$ Max profit = 65

$\Rightarrow$ Obj to be placed = 1 and 3 //